LAB REPORT: MUTEX AND SEMAPHORES

Course: Operating Systems

Name: Aryan Nautiyal **Roll Number:** 251210015

Aim

To simulate process synchronization problems, Producer–Consumer and Dining Philosophers problems—using mutex locks and semaphores, in order to prevent race conditions and deadlocks.

Theory

Mutex(mutual exclusion lock): is a synchronization primitive that ensures only one thread can enter a critical section at a time. A thread calls `pthread_mutex_lock` before accessing shared data and `pthread_mutex_unlock` after finishing. If another thread attempts to lock the same mutex while it is held, it blocks until the mutex is released.

Semaphore: a generalized counter maintained by the operating system. A counting semaphore initialized to N allows up to N threads to pass a `sem_wait` call without blocking; each `sem_post` increments the counter and wakes one waiting thread. A binary semaphore (initialized to 1) behaves similarly to a mutex but can also be used for signaling between threads.

Producer–Consumer problem: models a bounded buffer shared between producer and consumer threads. Two semaphores track the number of empty and full slots, while a mutex protects the buffer indices from concurrent modification. This pattern guarantees that producers block when the buffer is full and consumers block when it is empty, eliminating both race conditions and busy-waiting.

Dining Philosophers problem: involves N philosophers seated around a circular table, each needing two adjacent chopsticks to eat. The naïve approach—where every philosopher picks up the left chopstick first—creates a circular wait and deadlocks when all philosophers grab their left chopstick simultaneously. A standard prevention strategy breaks the symmetry by letting even-numbered philosophers pick up the left chopstick first while odd-numbered philosophers pick up the right one first, eliminating the circular-wait condition and guaranteeing progress.

Program

```c
#include <pthread.h>
#include <semaphore.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define SIZE 5
#define NUM_ITEMS 8

int buffer[SIZE];
int in = 0, out = 0, count = 0;
sem_t empty, full;
pthread_mutex_t mutex;

void print_buffer_visual(const char *action, int item) {
    printf("%-8s (Item %2d) | [ ", action, item);
    for (int i = 0; i < SIZE; i++) {
        int index = (out + i) % SIZE;
        if (i < count) printf("%2d ", buffer[index]);
        else           printf(" _ ");
    }
    printf("] (Count: %d/%d)\n", count, SIZE);
}
void *producer(void *arg) {
    for (int i = 1; i <= NUM_ITEMS; i++) {
        int item = rand() % 100 + 1;
        usleep((rand() % 400 + 200) * 1000);

        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = item;
        in = (in + 1) % SIZE;
        count++;
        print_buffer_visual("Produced", item);
```

```
        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    pthread_exit(NULL);
}
void *consumer(void *arg) {
    for (int i = 1; i <= NUM_ITEMS; i++) {
        usleep((rand() % 500 + 500) * 1000);

        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        out = (out + 1) % SIZE;
        count--;
        print_buffer_visual("Consumed", item);

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    pthread_exit(NULL);
}
void producer_consumer() {
    pthread_t prod_thread, cons_thread;

    sem_init(&empty, 0, SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    printf("----------- Producer-Consumer ----------\n");

    pthread_create(&prod_thread, NULL, producer, NULL);
    pthread_create(&cons_thread, NULL, consumer, NULL);
    pthread_join(prod_thread, NULL);
    pthread_join(cons_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);
}

#define NUM_P 5
sem_t chopsticks[NUM_P];

void *philosopher(void *num) {
    int id = (long)num;
    int left = id;
    int right = (id + 1) % NUM_P;

    printf("Philosopher %d is thinking...\n", id);
    sleep(1);

    if (id % 2 == 0) {
        sem_wait(&chopsticks[left]);
        sem_wait(&chopsticks[right]);
    } else {
        sem_wait(&chopsticks[right]);
        sem_wait(&chopsticks[left]);
    }

    printf("Philosopher %d is EATING using chopsticks %d and %d\n",
           id, left, right);
    sleep(1);

    sem_post(&chopsticks[left]);
```

```
    sem_post(&chopsticks[right]);

    printf("Philosopher %d has eaten and put chopsticks down.\n", id);
    pthread_exit(NULL);
}
void dining_philosopher() {
    pthread_t threads[NUM_P];

    printf("\n------------ Dining Philosophers ------------\n");

    for (int i = 0; i < NUM_P; i++)
        sem_init(&chopsticks[i], 0, 1);

    for (long i = 0; i < NUM_P; i++)
        pthread_create(&threads[i], NULL, philosopher, (void *)i);

    for (int i = 0; i < NUM_P; i++)
        pthread_join(threads[i], NULL);

    for (int i = 0; i < NUM_P; i++)
        sem_destroy(&chopsticks[i]);
}
int main() {
    producer_consumer();
    dining_philosopher();
    return 0;
}
```

Terminal Output

```
---------- Producer-Consumer ----------
Produced (Item 84) | [ 84  _  _  _  _ ] (Count: 1/5)
Produced (Item 87) | [ 84 87  _  _  _ ] (Count: 2/5)
Consumed (Item 84) | [ 87  _  _  _  _ ] (Count: 1/5)
Produced (Item 78) | [ 87 78  _  _  _ ] (Count: 2/5)
Produced (Item 16) | [ 87 78 16  _  _ ] (Count: 3/5)
Consumed (Item 87) | [ 78 16  _  _  _ ] (Count: 2/5)
Produced (Item 94) | [ 78 16 94  _  _ ] (Count: 3/5)
Consumed (Item 78) | [ 16 94  _  _  _ ] (Count: 2/5)
Produced (Item 36) | [ 16 94 36  _  _ ] (Count: 3/5)
Consumed (Item 16) | [ 94 36  _  _  _ ] (Count: 2/5)
Produced (Item 87) | [ 94 36 87  _  _ ] (Count: 3/5)
Produced (Item 93) | [ 94 36 87 93  _ ] (Count: 4/5)
Consumed (Item 94) | [ 36 87 93  _  _ ] (Count: 3/5)
Consumed (Item 36) | [ 87 93  _  _  _ ] (Count: 2/5)
Consumed (Item 87) | [ 93  _  _  _  _ ] (Count: 1/5)
Consumed (Item 93) | [  _  _  _  _  _ ] (Count: 0/5)

------------ Dining Philosophers ------------
Philosopher 0 is thinking...
Philosopher 1 is thinking...
Philosopher 2 is thinking...
Philosopher 3 is thinking...
Philosopher 4 is thinking...
Philosopher 0 is EATING using chopsticks 0 and 1
Philosopher 2 is EATING using chopsticks 2 and 3
Philosopher 0 has eaten and put chopsticks down.
Philosopher 2 has eaten and put chopsticks down.
Philosopher 1 is EATING using chopsticks 1 and 2
Philosopher 4 is EATING using chopsticks 4 and 0
Philosopher 1 has eaten and put chopsticks down.
Philosopher 4 has eaten and put chopsticks down.
Philosopher 3 is EATING using chopsticks 3 and 4
Philosopher 3 has eaten and put chopsticks down.
```